

Reviewer Pack — Coboundary Spectral Gap Lower-Bounds DPLL Tree Size on Tseitin

Entry 5ca5616545f4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 11:29:03 UTC

Coboundary Spectral Gap Lower-Bounds DPLL Tree Size on Tseitin

Entry ID: 5ca5616545f4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 11:29:03 UTC

1. Conjecture

Field A (mathematical branch): Linial-Meshulam-Wallach high-dimensional simplicial coboundary expansion (spectral gap of the up-Laplacian on 1-cochains) **Field B** (complexity object): DPLL back-track count (tree-like resolution refutation size) for Tseitin formulas on 3-regular graphs

Statement:

Let G be a connected 3-regular graph on n vertices and $c:V(G) \rightarrow \{0,1\}$ have odd Hamming weight, so $Ts(G,c)$ is unsatisfiable. Let $X(G)$ be the 2-complex whose 0-simplices are edges of G , 1-simplices are pairs of G -edges sharing a vertex, and 2-simplices are the (unique) triples of G -edges meeting at each vertex, and let $\lambda_1(G)$ be the smallest nonzero eigenvalue of the real 1-up-Laplacian $\delta_1^* \delta_1$ on $C^1(X(G); \mathbb{R})$. Then there is an absolute constant $\kappa > 0$ such that for every such (G,c) , $\log_2(\min \text{DPLL backtracks to refute } Ts(G,c)) \geq \kappa \cdot \lambda_1(G) \cdot \sqrt{n}$; a single (G,c) violating this bound by $\geq 20\%$ at $n \geq 18$ falsifies the conjecture.

Rationale (proposer's reasoning):

Graph-expansion lower bounds on Tseitin (Ben-Sasson-Wigderson) capture only 1-dimensional vertex flow, but DPLL must propagate parity contradictions through local 'around-a-vertex' triples — exactly the 2-simplices of $X(G)$; the Linial-Meshulam up-Laplacian eigenvalue is the canonical spectral surrogate for higher-order Cheeger expansion of that triple-structure. If true, λ_1 provides a polynomial-time computable structural certificate, complementary to vertex expansion, for tree-resolution hardness, and being a property of the formula's incidence geometry (not its truth table) it sidesteps natural proofs, relativization, and algebrization.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a3e18da4df2d1651

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 210 instances (30 random 3-regular graphs $\times$ 7 sizes $n \in \{8, \dots, 20\}$, odd-weight c), run deterministic DPLL with unit prop + most-frequent-literal branching to count backtracks B , and compute $\lambda \uparrow(G)$ via `numpy.linalg.eigh`. Conjecture is SUPPORTED iff OLS slope of $\log_2(B)$ on $\lambda \uparrow \cdot \sqrt{n}$ is >0 AND $\geq 80\%$ of instances satisfy $\log_2(B) \geq 0.05 \cdot \lambda \uparrow \cdot \sqrt{n}$; FALSIFIED iff any instance with $n \geq 18$ has $\log_2(B) < 0.8 \cdot (0.05 \cdot \lambda \uparrow \cdot \sqrt{n})$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.88	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Tseitin formulas tree-like resolution lower bounds expander 3-regular-high-dimensional expander coboundary expansion up-Laplacian spectral gap proof complexity - simplicial complex Linial Meshulam cohomological expansion Tseitin DPLL resolution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3_regular_graph(n):
        if n % 2 != 0 or n < 4:
            return None
        vertices = list(range(n))
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if len(edges) == (n * (n - 1)) // 6:
                    break
                if random.choice([True, False]):
                    edges.append((i, j))
        return vertices, edges

    def compute_lambda_up(X_G):
        m = len(X_G)
        D = [[0] * m for _ in range(3)]
```

```

    for i in range(m):
        u, v = X_G[i]
        D[0][u], D[0][v] += 1
        D[1][i] += 2
        D[2][i] += 1

    L1 = [[D[2][i] - D[1][j] if (i // 3 == j // 3 and i % 3 != j % 3) else 0 for j in range(m)]

    # Compute eigenvalues of L1
    eigvals = []
    for i in range(m):
        v = [D[2][i] - D[1][j] if (i // 3 == j // 3 and i % 3 != j % 3) else 0 for j in range(m)]
        eigval = sum(v[j] * v[k] for j in range(m) for k in range(j + 1, m)) / (2 * sum(v[j]**2 for j in range(m)))
        eigvals.append(eigval)

    return min(eigvals)

def dp11(G, c):
    stack = []
    backtrack_count = 0
    while True:
        if not stack:
            return backtrack_count

        u = next((v for v in G if c[v] == 'u'), None)
        if u is None:
            return backtrack_count

        stack.append(u)
        c[u] = '1'

        if all(c[v] != '0' for v in G):
            return backtrack_count

        for v in G:
            if c[v] == 'u':
                c[v] = '0'
                stack.append(v)
                backtrack_count += 1
                break

n_values = [8, 10, 12, 14, 16, 18, 20]
results = []

for n in n_values:
    G = generate_3_regular_graph(n)
    if G is None:
        continue

    vertices, edges = G
    X_G = [(u, v) for u, v in edges]

    c = {v: 'u' for v in vertices}
    backtrack_count = dp11(G, c)

    lambda_up = compute_lambda_up(X_G)

```

```

        results.append({
            "n": n,
            "backtrack_count": backtrack_count,
            "lambda_up": lambda_up
        })

    if not results:
        return {
            "metric_name": "log2(backtrack_count)",
            "metric_value": 0.0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "graph_generation_failed"
        }

    backtrack_counts = [r["backtrack_count"] for r in results]
    lambda_ups = [r["lambda_up"] for r in results]
    slopes = [(math.log2(b) - math.log2(a)) / (b - a) if b != a else float('inf') for a, b in zip(backtrack_counts, lambda_ups)]

    mean_slope = sum(slopes) / len(slopes)
    intercepts = [math.log2(b) - mean_slope * b for b in backtrack_counts]
    min_ratio = min(math.log2(b) / (lambda_up * math.sqrt(n)) for n, b, lambda_up in zip(results[0]['n'], results[0]['backtrack_count'], results[0]['lambda_up']))

    return {
        "metric_name": "log2(backtrack_count)",
        "metric_value": mean_slope,
        "instances_tested": len(results),
        "conjecture_holds": mean_slope > 0 and min_ratio >= 0.05,
        "counterexample": "" if mean_slope > 0 else f"n={results[0]['n']}, backtrack_count={results[0]['backtrack_count']}, lambda_up={results[0]['lambda_up']}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_slope = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slope} std=0.0 support_fraction=1.0")
    elif any(r["n"] >= 18 and r["backtrack_count"] < 0.8 * (0.05 * r["lambda_up"] * math.sqrt(r["n"])) for r in results):
        first_failing_seed = next(r["seed"] for r in results if r["n"] >= 18 and r["backtrack_count"] < 0.8 * (0.05 * r["lambda_up"] * math.sqrt(r["n"])))
        print(f"RESULT: FALSIFIED counterexample='n≥18 backtrack_count too low' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aee00bc8.py", line 144, in <module>
    result = run_trial(seed)
            ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aee00bc8.py", line 111, in run_trial
    lambda_up = compute_lambda_up(X_G)
               ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aee00bc8.py", line 72, in compute_lambda_up
    L1 = [[D[2][i] - D[1][j] if (i // 3 == j // 3 and i % 3 != j % 3) else 0 for j in range(m)] for i in range(m)]
          ~~~~~^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in compute_lambda_up before any instances were evaluated, so no data was produced to assess the pre-registered support or falsification conditions. | next: Fix the L1 up-Laplacian construction (correct indexing/boundary maps on the 2-complex $X(G)$) and re-run the 210-instance protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	97091
2	preregistration	claude_max	opus	0	0	6319
3	novelty	claude_max	opus	0	0	4692
4	novelty	claude_max	opus	0	0	7380
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26629
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19454
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19694
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16428
9	judge	claude_max	opus	0	0	3886

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 201573 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/5ca5616545f4.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5ca5616545f4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5ca5616545f4.tar.gz` (if generated)